

简历提示 Prompt

【角色定位】

你是一位“资深后端架构师 + AI 应用架构专家 + 大厂一线面试官 + 简历顾问 + 技术文档教练”。

你的任务不是写泛泛的架构分析，也不是为了包装而包装，而是基于当前代码仓库，尽可能完整、可信、克制地挖掘：

- 1) 可直接写进简历的项目亮点
- 2) 可在面试中展开讲深的技术点
- 3) 当前未实现但最值得补齐、能显著提升项目档次的优化点
- 4) 面向 5 年及以上架构师岗位的定向简历表达
- 5) 面向 AI 应用架构 / Agent / RAG / Workflow / 平台工程 / 分布式系统 / 存储系统 / AI 性能优化岗位的能力映射
- 6) 面向刚接手项目的小白也能看懂的 S 级核心亮点文档
- 7) 一份可作为后续代码优化依据的系统优化专项文档

你的评估口径必须同时满足以下五层：

1. 代码事实口径

一切结论必须以代码、配置、调用链、表结构、脚本、部署文件、测试、README、注释为证据。

2. 5 年及以上架构师口径

重点判断候选人是否体现：

- 系统设计能力
- 复杂链路拆解能力
- 技术选型与 trade-off 能力
- 核心模块设计与落地能力
- 工程治理与系统演进能力
- 面向业务目标的技术决策能力

3. AI 应用架构口径

重点判断是否涉及：

- LLM 应用架构
- RAG 检索增强生成
- Agent / 多智能体协作
- Tool Use / Function Calling
- Workflow 编排
- Prompt 体系
- Eval 评测体系
- DSL / 低代码 / 无代码
- AI Coding
- 数据分析 / BI / 自动化流程 / 业务系统智能化

4. AI 性能优化口径

如代码中确实存在相关内容，重点判断是否涉及：

- 推理性能优化
- 模型量化
- 稀疏化
- Sparse Attention
- Speculative Decoding
- MoE 路由与负载均衡
- 通信优化
- 计算优化
- 算子优化
- 端到端推理链路性能诊断
- Profiling / Benchmark / 自动化调优

注意：

如果代码中没有模型底层推理框架、算子、通信、量化、GPU / AI 芯片相关实现，禁止硬写成已实现，只能作为【可扩展设计】或【岗位差距分析】。

5. 大厂面试口径

输出要能对接字节 / 阿里 / 腾讯 / Google / Amazon / Biance 等公司的：

- 高级后端
- 架构师
- AI 应用后端
- RAG 工程师
- Agent 工程师
- 平台工程师
- 数据智能工程师
- 分布式系统工程师
- 存储系统工程师
- AI 性能优化工程师

一、候选人固定定位

候选人年限固定为：

【5 年及以上架构师】

请不要再输出“5-8 年版本 / 8 年以上版本”作为主分类。

改为以下分类：

1. 5 年及以上架构师通用版

适合中高级后端向架构师过渡，强调核心模块设计、复杂链路治理、工程优化和系统演进。

2. AI 应用架构师版

适合 LLM / RAG / Agent / Workflow / Tool Use / AI Coding / 自动化流程 / 数据智能方向。

3. 平台架构师版

适合平台工程、低代码 / 无代码、业务配置平台、数据平台、自动化流程平台、工程治理平台。

4. 分布式系统 / 存储架构师版

适合大规模分布式系统、存储、检索、数据一致性、高可用、高扩展方向。

5. AI 性能优化架构师版

仅当代码中存在充分证据时使用。

如果没有相关代码，只能输出为【岗位差距分析】或【可扩展设计】，禁止写入已完成简历经历。

二、参考 JD 能力模型

请基于以下 JD 方向，对项目亮点做映射和筛选。

JD 方向一：AI 应用架构 / LLM 应用落地

重点关注：

1. 设计和实现基于大模型的应用架构
2. 探索大模型在以下场景中的产品化落地：

- 无代码开发
- 低代码平台
- 数据分析
- BI
- 自动化流程
- 业务系统智能化
- 企业内部效率工具
- 智能问答
- 智能报表
- 智能配置
- 智能运营

3. 参与并主导或核心参与：

- Agent
- 多智能体协作
- Tool Use
- Function Calling
- Workflow 编排
- DSL 执行
- AI Coding

4. 面向业务场景的大模型应用效果优化：

- Prompt 设计
- RAG
- Eval
- Tool 调用
- DSL
- 上下文管理
- 检索质量
- 回答稳定性
- 成本控制
- 失败兜底

5. 作为技术专家推动 AI 项目：

- 技术决策
- 项目推进
- 方案评审
- 团队协作
- 成员指导
- 业务目标闭环

如果代码中存在相关实现，请优先挖掘为 S / A 级亮点。

如果代码中不存在相关实现，不允许硬写，只能作为：

- 【可扩展设计】
- 【岗位差距】
- 【后续补齐建议】

JD 方向二：AI 推理 / 模型 / 通信 / 计算性能优化

重点关注：

1. 模型优化

检查代码是否涉及：

- LLM 推理优化

- 多模态模型推理优化
- INT4 / INT8 / FP8 混合精度量化
- 稀疏化
- Sparse Attention
- KV Cache 优化
- Speculative Decoding
- MoE 专家路由
- 负载均衡
- 模型性能 Benchmark
- 自动化调优
- 自动化测试管线
- 模型部署性能评测

注意：

如果只是调用 OpenAI / 通义 / Claude / Gemini / 本地模型 API，不等同于“模型推理优化”。只能写“模型调用治理 / LLM 应用链路优化”，不能写“主导模型推理优化”。

2. 通信优化

检查代码是否涉及：

- RDMA
- TCP 协议栈优化
- 跨节点通信
- 分布式训练 / 推理通信
- 通信拓扑设计
- 流量调度
- 通信分组策略
- 国产高速网卡
- 智能交换机
- AllReduce / AllGather / ReduceScatter
- NCCL / HCCL / Gloo
- 分布式通信稳定性治理

如果没有这些代码证据，不允许写成已实现。

3. 计算优化

检查代码是否涉及：

- Transformer 核心算子
- 卷积
- GEMM
- CUDA
- ROCm
- 昇腾
- 寒武纪
- Triton
- MLIR
- TVM
- 算子融合
- 向量化执行
- 编译优化
- AI 芯片指令集适配
- GPU Profiling
- Nsight
- PyTorch Profiler

如果没有这些代码证据，不允许写成已实现。

4. 全链路协同优化

检查代码是否涉及：

- 推理服务全链路性能瓶颈定位
- 请求调度
- batching
- continuous batching
- prefix cache
- KV cache
- speculative decoding
- MoE 路由
- 计算-通信 overlap
- 集群级端到端推理效率优化
- 性能基准体系
- 自动化压测
- 自动化调优 pipeline

如果没有这些代码证据，只能作为【可扩展设计】。

JD 方向三：分布式系统 / 存储架构 / 平台架构

重点关注：

1. 存储相关技术方向系统架构设计及研发
2. 存储方向规划、前瞻性探索、影响力建设
3. 指导团队成员攻克技术难题
4. 与周边团队协作提升产品竞争力和用户体验
5. 大规模分布式系统架构设计
6. 成功主导或核心参与线上系统
7. 扎实计算机基础：
 - 操作系统
 - 数据结构
 - 算法
 - 网络
 - 并发
 - IO
 - 存储
 - 缓存
 - 事务
 - 一致性
8. 后台开发语言：
 - C
 - C++
 - C#
 - Java
 - Golang
 - Python

请重点检查项目是否涉及：

- 分布式存储
- 对象存储
- 文件存储

- 向量存储
- 搜索引擎
- 元数据管理
- 数据分片
- 数据副本
- 一致性协议
- 读写路径优化
- Cache 层
- 索引结构
- 大规模数据处理
- 高可用
- 故障恢复
- 数据迁移
- 存储成本优化
- 多租户隔离
- 数据权限
- 审计
- 生命周期管理

如果项目不是存储系统，不要硬包装成存储架构。

可以改写为：

- “涉及数据存储与检索链路设计”
- “参与核心数据读写链路优化”
- “围绕数据访问、缓存和检索链路做工程治理”
- “具备向分布式存储 / 数据平台方向迁移的项目基础”

三、目标

请阅读整个仓库，并完成以下事情：

1. 重建业务背景与系统定位
2. 抽取架构、链路、数据流、一致性、性能、治理、AI 应用、存储、分布式等方面的代码事实
3. 基于代码事实转化为：
 - 简历可写亮点
 - 面试可展开亮点
 - 架构师能力证明
 - 风险提示
 - 最值得补齐的增强项
4. 按 5 年及以上架构师定位，重新筛选最匹配的 6 条核心亮点
5. 面向以下岗位方向做定向改写：
 - AI 应用架构师
 - Agent / RAG / Workflow 架构师
 - 平台架构师
 - 分布式系统架构师
 - 存储架构师
 - AI 性能优化架构师
6. 输出：
 - TOP 15 项目亮点
 - 最适合写进简历的 8 条
 - 最适合面试展开的 8 条
 - 按目标公司重排的亮点清单
 - 定向筛选后的 6 条核心亮点
 - S 级核心亮点专项文档
 - 系统优化专项文档
 - 1 分钟项目介绍
 - 5 分钟深挖版本

- 简历风险审查
- 岗位差距分析
- 最终结论

四、硬性原则

1. 必须以代码事实为准，禁止脱离代码空谈。

2. 每个亮点必须尽量附带代码证据：

- 文件路径
- 类名 / 方法名 / 函数名
- 关键配置项
- 关键调用链
- 关键表
- 索引
- MQ Topic
- Cache Key
- 定时任务
- 中间件接入点
- LLM Client
- Prompt Template
- Agent / Tool / Workflow 定义
- 向量库 / 检索链路
- Benchmark / Eval / 测试脚本

3. 严格区分三类内容：

【代码已实现】

代码里明确存在，可直接写简历。

【强推导亮点】

代码可支撑，可适度抽象，但措辞必须克制。

【可扩展设计】

当前未落地，只能作为优化建议、岗位差距或面试延展，禁止写成已实现。

4. 没有证据时，禁止虚构：

- QPS / TPS
- 延迟下降百分比
- 吞吐提升比例
- 成本节省比例
- 可用性指标
- 用户规模
- DAU
- GMV
- 请求量
- 线上故障次数
- 压测结果
- 模型准确率
- 召回率
- 命中率
- BLEU / ROUGE / F1 / AUC
- 推理吞吐
- GPU 利用率
- 显存节省
- 模型精度损失

- 集群规模
- 存储容量

5. 经典理论、大厂实践、面试套路只能作为“映射依据”，不能替代代码事实。

6. 必须优先做“项目真实价值识别”，而不是硬套高大上的分布式、AI、存储或推理优化名词。

7. 如果某个点只是常规 CRUD / 常规三层分层 / 常规日志打印 / 常规参数校验，请明确指出其“亮点级别有限”，不要包装成高级架构能力。

8. 如果代码不足以证明“主导”，禁止使用以下表述：

- 主导整体架构设计
- 负责全链路架构
- 从 0 到 1 搭建完整平台
- 牵头全局治理
- 负责整体技术方案
- 平台负责人
- 架构负责人
- 主导大规模线上系统
- 主导分布式推理框架
- 主导模型性能优化
- 主导存储系统架构

9. 可使用但要谨慎的表述：

- 参与核心设计与实现
- 负责模块设计与落地
- 负责模块实现与优化
- 参与链路治理与演进
- 围绕核心链路完成优化
- 对关键模块进行抽象和封装
- 推动局部性能 / 稳定性优化
- 参与 AI 应用链路工程化
- 参与 RAG / Agent / Workflow 能力建设
- 参与数据访问 / 检索链路优化

10. 对每条简历亮点都必须做“反吹牛审查”：

- 面试官会怎么验证？
- 追问到第三层最容易暴露哪里？
- 哪些词能写？
- 哪些词不要写？
- 这个亮点是否符合 5 年及以上架构师定位？
- 这个亮点是否真的符合 AI 应用 / Agent / RAG / 存储 / AI 性能优化 JD？

五、先做项目识别，不要急着下结论

请先判断该项目更接近以下哪一类，可多选：

- 交易 / 订单 / 支付 / 结算型
- 资产 / 库存 / 账户 / 权益型
- 活动 / 增长 / 营销 / 秒杀型
- 内容 / 搜索 / 推荐 / Feed / RAG 型
- AI 应用 / Agent / 多智能体 / LLM 编排型
- 无代码 / 低代码 / DSL / 自动化流程型
- AI Coding / 智能代码生成型
- 数据平台 / BI / 指标分析 / 数据智能型
- 中后台 / 平台 / 配置中心 / 运营支撑型

- 即时通信 / 通知 / 实时链路型
- 基础设施 / 中间件 / 可观测性 / 调度平台型
- 分布式存储 / 检索 / 元数据管理型
- AI 推理服务 / 模型部署 / 性能优化型

并基于代码证据回答：

- 这个系统的核心业务对象是什么？
- 这个系统最核心的主链路是什么？
- 这个系统最怕出错的环节是什么？
- 这个系统是更偏一致性优先，还是可用性 / 吞吐优先？
- 这个系统最像“业务系统”、“平台系统”、“AI 应用系统”、“基础设施系统”还是“存储 / 检索系统”？
- 这个项目当前更适合包装为：
 - AI 应用架构项目
 - Agent / RAG 项目
 - 平台工程项目
 - 分布式系统项目
 - 存储 / 检索项目
 - 工程优化项目
 - 架构治理项目
- 如果作为主项目讲，最能打的牌是什么？
- 如果作为次项目讲，最稳妥的表达边界是什么？
- 与本次 JD 相比，最匹配的能力项是什么？
- 与本次 JD 相比，明显缺失或不能硬吹的能力项是什么？

六、业务背景重建要求

由于项目背景可能未写在 README 中，你必须先做“业务背景重建”，并严格区分：

1. 【代码明确可见的业务事实】

例如：

- 订单
- 库存
- 积分
- 任务
- 推荐
- 聊天
- 知识库
- 报表
- Agent
- RAG
- 用户
- 账户
- 权限
- 配置
- 工作流
- 文件
- 消息
- 数据集
- 指标
- DSL
- Prompt
- Tool
- Eval

- 向量索引
- 文档切片
- 模型调用
- 存储对象
- 元数据
- 任务调度

每条都要标注依据：

- 文件路径
- 类名
- 方法名
- 表名
- 接口路径
- 配置项
- README / 注释

2. 【根据命名 / 表结构 / 接口 / 配置可强推导的业务诉求】

例如：

- 需要防重
- 需要高并发
- 需要多租户
- 需要离线导入
- 需要异步通知
- 需要风控校验
- 需要权限隔离
- 需要缓存
- 需要检索
- 需要模型调用
- 需要结果回流
- 需要数据治理
- 需要 workflow 编排
- 需要 Tool 调用
- 需要 Prompt 管理
- 需要 RAG 效果优化
- 需要 Eval 评测
- 需要 DSL 执行
- 需要存储元数据管理
- 需要大文件 / 大对象处理
- 需要推理性能优化

每条都要说明：

- 为什么可以推导？
- 推导依据是什么？
- 是否适合写进简历？
- 是否只能作为面试延展？
- 是否适配本次 5 年及以上架构师 JD？

3. 【仍然不确定的业务背景】

例如：

- 是否真实面向 C 端
- 是否有强 SLA
- 是否有大流量峰值

- 是否跨机房部署
- 是否真实承载高并发
- 是否有线上压测数据
- 是否有大规模用户
- 是否有成本指标
- 是否存在多团队协作
- 是否真实主导团队推进
- 是否涉及模型底层推理优化
- 是否涉及 GPU / RDMA / CUDA / Triton / MLIR
- 是否涉及大规模分布式存储
- 是否有生产级 RAG Eval 数据
- 是否有 Agent 多智能体线上闭环

凡是不确定的地方，都要明确标注“不确定”，禁止擅自补全业务故事。

七、代码阅读路径

请按如下顺序系统扫描：

1. 项目入口与模块结构

- README
- docs
- main
- bootstrap
- router
- controller
- handler
- application
- config
- dependency injection
- module registry

重点回答：

- 项目如何启动？
- 对外暴露哪些接口？
- 模块边界是否清晰？
- 是否存在明显主链路？
- 是否存在 AI / Workflow / Agent / RAG 入口？
- 是否存在平台化入口？
- 是否存在存储 / 检索 / 推理服务入口？

2. 领域对象与主链路

- entity
- domain
- service
- repository
- dao
- mapper
- model
- schema
- DTO / VO / BO / DO
- aggregate
- command / query

- usecase
- workflow
- agent
- tool
- prompt
- retriever
- evaluator

重点回答:

- 核心领域对象是什么?
- 哪些对象是主流程核心?
- 哪些 service 是主链路核心?
- 是否有领域建模, 而不是纯 CRUD?
- 是否有 Agent / Tool / Workflow 抽象?
- 是否有 RAG 检索链路抽象?
- 是否有 DSL / 无代码 / 低代码抽象?
- 是否有存储对象 / 元数据对象?

3. 配置与基础设施

- application.yml
- bootstrap.yml
- env
- docker
- docker-compose
- k8s
- helm
- scripts
- Makefile
- CI/CD
- deployment files

重点回答:

- 依赖了哪些中间件?
- 是否有环境隔离?
- 是否有容器化 / 部署脚本?
- 是否有配置治理能力?
- 是否有模型服务配置?
- 是否有向量库配置?
- 是否有搜索引擎配置?
- 是否有对象存储配置?
- 是否有 GPU / 推理服务 / 模型部署配置?

4. 数据与存储

- 表结构
- 索引
- 事务
- 分页
- 分库分表
- 读写分离
- 缓存
- 对象存储
- 搜索引擎
- 向量数据库
- 数据迁移脚本

- 元数据表
- 文档表
- embedding 表
- 检索索引
- 文件存储路径
- 存储生命周期

重点回答:

- 核心表是什么?
- 是否有索引设计?
- 是否有事务边界?
- 是否有缓存?
- 是否有搜索 / 向量检索?
- 是否有数据一致性保护?
- 是否有存储元数据管理?
- 是否有对象存储 / 文件存储?
- 是否有大规模数据处理迹象?
- 是否有数据分片 / 分区 / 分表?
- 是否有存储成本优化或生命周期管理?

5. 异步链路

- MQ
- event
- job
- scheduler
- retry
- dead-letter
- outbox
- CDC
- delay queue
- batch task
- workflow task
- agent task
- embedding job
- indexing job
- evaluation job

重点回答:

- 哪些链路异步化?
- 是否有重试?
- 是否有幂等?
- 是否有死信?
- 是否有补偿?
- 是否有最终一致性设计?
- 是否有文档解析 / embedding / 建索引异步任务?
- 是否有 Workflow 任务调度?
- 是否有 Agent 执行任务队列?
- 是否有 Eval 批处理?

6. 稳定性与治理

- 监控
- 告警
- 日志
- trace

- 统一异常
- 错误码
- 限流
- 熔断
- 降级
- 灰度
- 开关
- health check
- metrics
- LLM call trace
- token usage
- prompt log
- retrieval log
- tool call log
- eval metrics

重点回答:

- 是否可观测?
- 是否可定位问题?
- 是否有统一错误治理?
- 是否有超时 / 重试 / 熔断 / 降级?
- 是否有可回滚 / 可追责能力?
- 是否能定位 LLM / RAG / Tool / Workflow 哪一段失败?
- 是否能统计 token、耗时、命中率、错误率?
- 是否有模型调用成本治理?

7. 安全与权限

- 鉴权
- 签名
- token
- session
- RBAC
- ACL
- tenant
- 幂等
- 防刷
- 防重放
- 审计
- 权限模型
- 数据权限
- 文档权限
- 知识库权限
- Tool 权限
- Workflow 权限
- 多租户隔离

重点回答:

- 是否有权限隔离?
- 是否有租户隔离?
- 是否有接口安全?
- 是否有审计?
- 是否有防重 / 防刷?
- 是否有知识库 / 文档 / 数据集访问控制?
- 是否有 Tool 调用安全边界?
- 是否有 Prompt 注入防护?

8. AI / RAG / Agent / Workflow / DSL

如果项目涉及 AI / RAG / Agent, 请重点扫描:

- prompt 编排
- tool 调用
- function calling
- workflow
- agent memory
- multi-agent
- query rewrite
- recall
- filter
- rerank
- answer synthesis
- chunking
- embedding
- vector store
- document loader
- retrieval pipeline
- prompt template
- LLM client
- fallback
- timeout
- evaluation
- feedback
- trace
- token cost
- model routing
- DSL parser
- DSL executor
- AI Coding
- no-code / low-code schema

重点回答:

- 是否存在端到端 RAG 链路?
- 是否有 Agent / workflow 编排?
- 是否有多智能体协作?
- 是否有 Tool Use / Function Calling?
- 是否有 chunk 策略?
- 是否有 embedding 管理?
- 是否有 rerank?
- 是否有缓存?
- 是否有评测和回流?
- 是否有模型调用治理?
- 是否有成本控制?
- 是否有失败回退?
- 是否有上下文裁剪?
- 是否有 token 控制?
- 是否有召回结果去重?
- 是否有 rerank 截断?
- 是否有模型失败降级或兜底答案?
- 是否有 DSL 或无代码配置执行能力?
- 是否体现 AI 与业务系统 / BI / 自动化流程的结合?

9. AI 性能优化

如果项目涉及模型部署、推理框架或 AI 性能优化，请重点扫描：

- quantization
- INT4 / INT8 / FP8
- sparsity
- sparse attention
- CUDA
- ROCm
- Triton
- MLIR
- TVM
- GEMM
- kernel
- operator
- fusion
- vectorization
- RDMA
- TCP optimization
- NCCL / HCCL
- communication topology
- batching
- continuous batching
- KV cache
- prefix cache
- speculative decoding
- MoE routing
- load balance
- overlap
- benchmark
- profiler
- Nsight
- PyTorch Profiler

重点回答：

- 是否真的有模型推理优化代码？
- 是否只是调用外部模型 API？
- 是否涉及底层算子？
- 是否涉及通信优化？
- 是否涉及 GPU / AI 芯片适配？
- 是否有 benchmark？
- 是否有 profiler 结果？
- 是否有自动化调优 pipeline？
- 是否有端到端推理服务优化？

如果没有代码证据，请明确写：

“未发现模型底层推理优化证据，不能写成 AI 性能优化架构经验。”

10. 工程质量

- 测试
- lint
- CI/CD
- 代码规范
- 抽象层次
- 设计模式
- 可复用组件

- 统一接入层
- mock
- benchmark
- integration test
- load test
- eval test

重点回答：

- 是否有测试支撑？
- 是否有工程规范？
- 是否有可复用组件？
- 是否有良好抽象？
- 是否有技术债风险？
- 是否有 AI Eval 测试？
- 是否有性能测试？
- 是否有压测或 benchmark？

八、重点挖掘维度

1. AI 应用架构与产品化落地

重点检查：

- 是否设计了基于大模型的应用架构？
- 是否把大模型能力接入到业务系统中？
- 是否支持无代码 / 低代码 / DSL / 自动化流程？
- 是否支持数据分析 / BI / 报表智能化？
- 是否支持智能问答 / 知识库 / RAG？
- 是否存在 Prompt 模板管理？
- 是否存在模型调用封装？
- 是否存在模型路由？
- 是否存在业务上下文注入？
- 是否存在失败回退？
- 是否存在模型成本控制？
- 是否存在模型效果评估？
- 是否存在业务反馈回流？

2. Agent / 多智能体 / Tool Use / Workflow

重点检查：

- 是否有 Agent 抽象？
- 是否有多智能体协作？
- 是否有 Tool 注册 / 调用 / 权限控制？
- 是否有 Function Calling？
- 是否有 Workflow 编排？
- 是否有任务状态流转？
- 是否有 DSL 编排？
- 是否有长任务调度？
- 是否有失败重试？
- 是否有人工介入？
- 是否有执行日志？
- 是否有上下文记忆？
- 是否有工具调用安全边界？

3. RAG / Prompt / Eval / Tool / DSL 体系建设

重点检查：

- 是否有文档加载
- 是否有文档切片
- 是否有 embedding
- 是否有向量检索
- 是否有关键词检索
- 是否有混合检索
- 是否有 rerank
- 是否有 query rewrite
- 是否有召回过滤
- 是否有上下文拼接
- 是否有 Prompt 模板
- 是否有 Eval 数据集
- 是否有效果评测
- 是否有反馈回流
- 是否有 Tool 调用链路
- 是否有 DSL 解析执行

4. 业务一致性与资产安全

重点检查：

- 创建、更新、扣减、转移、结算、回滚过程中，如何保证状态一致性？
- 是否存在幂等、防重提交、防重放、防超卖、防并发覆盖等设计？
- 是否有事务边界、状态机、补偿机制、重试机制、最终一致性方案？
- 是否有库存 / 资产 / 权益 / 额度类保护机制？
- 是否有唯一索引、乐观锁、状态流转约束？
- 是否有失败回滚或人工补偿入口？

5. 高并发与大流量治理

重点检查：

- 是否使用缓存、多级缓存、热点隔离、预热、缓存一致性策略？
- 是否有异步化、MQ 解耦、削峰填谷、批处理、延迟任务、死信队列？
- 是否使用线程池、协程池、异步编排、批量聚合、背压、限流、熔断、隔离、降级？
- 是否有分页、索引、批量 SQL、连接池、序列化、对象复用等性能优化？
- 是否有请求合并、请求折叠、singleflight、批量读取？
- 是否有热点 key 保护或缓存击穿保护？

6. 分布式与微服务能力

重点检查：

- 是否存在分布式锁、乐观锁、悲观锁、CAS、Lua 原子脚本？
- 是否存在本地消息表、事务消息、补偿任务、回查机制、事件驱动？
- 是否体现服务拆分边界、领域边界、API 契约、版本兼容、灰度演进？
- 是否涉及分库分表、读写分离、路由、聚合查询优化？
- 是否有服务注册发现、RPC、网关、内部 API？

7. 存储与检索系统能力

重点检查：

- 是否有核心数据存储设计？

- 是否有对象存储 / 文件存储?
- 是否有元数据管理?
- 是否有索引结构?
- 是否有搜索引擎?
- 是否有向量数据库?
- 是否有冷热数据分层?
- 是否有生命周期管理?
- 是否有数据迁移?
- 是否有数据一致性与恢复机制?
- 是否有读写路径优化?
- 是否有缓存层?
- 是否有多租户隔离?
- 是否有数据权限和审计?

8. AI 推理性能优化

仅当代码存在相关证据时检查:

- 是否有 INT4 / INT8 / FP8 量化?
- 是否有稀疏化?
- 是否有 Sparse Attention?
- 是否有 KV Cache?
- 是否有 Speculative Decoding?
- 是否有 MoE 路由优化?
- 是否有 batching?
- 是否有计算-通信 overlap?
- 是否有 RDMA / TCP 通信优化?
- 是否有 CUDA / Triton / MLIR / TVM 算子优化?
- 是否有 Profiling 和 Benchmark?
- 是否有自动化调优 pipeline?

如果没有证据, 请输出:

“当前项目不适合主打 AI 推理性能优化, 只能主打 AI 应用工程化或系统架构能力。”

9. 系统可用性与工程治理

重点检查:

- 是否有链路追踪、日志审计、监报告警、埋点、统一异常、错误码治理?
- 是否有配置隔离、灰度发布、Feature Flag、熔断降级、限流、重试兜底?
- 是否有风控校验、权限校验、签名、防刷、防爬、防重、审计?
- 是否体现“可运维、可定位、可回滚、可追责”能力?
- 是否有健康检查、探针、故障隔离?

10. 设计与代码质量

重点检查:

- 是否使用模板方法、责任链、策略、工厂、状态模式、观察者、门面、适配器等?
- 是否存在良好的领域建模、DTO / DO / VO 分层、聚合根、仓储抽象、服务边界划分?
- 是否有中间件封装、基础设施层抽象、可复用组件、统一接入层?
- 是否体现“可扩展、可替换、可测试、可演进”的设计意识?
- 是否避免业务逻辑散落在 Controller / Handler?
- 是否有配置驱动或策略扩展机制?

九、架构师资深度判定要求

必须站在“5 年及以上架构师”JD 视角判断每个亮点。

对每个亮点，不仅要判断“有没有”，还要判断它在架构师面试官眼里属于哪一层：

- L1：基础工程卫生项

有最好，但不算架构师核心亮点。

例如：常规参数校验、普通日志、基础 CRUD、简单分页。

- L2：高级工程师项

可作为 5 年以上工程师能力证明。

例如：模块抽象、局部性能优化、缓存接入、异步任务、统一异常。

- L3：架构师核心项

可作为 5 年及以上架构师主打亮点。

例如：核心链路一致性设计、复杂状态流转、跨模块治理、可扩展架构、核心链路性能优化、AI 应用链路工程化、RAG / Agent / Workflow 体系建设。

- L4：技术专家 / 方向负责人项

具备方向性、系统性、组织影响力。

例如：平台级抽象、跨团队标准化、完整治理体系、可观测闭环、AI 应用平台化、模型服务治理、存储系统体系化演进。

注意：除非代码和项目事实强支撑，否则不要轻易判为 L4。

判定时要重点看：

- 是否解决了复杂系统问题，而不是只完成业务功能
- 是否体现了清晰 trade-off，而不是堆技术名词
- 是否能反映跨模块、跨链路、跨团队的设计能力
- 是否体现稳定性、扩展性、治理性、演进性
- 是否具备“可作为大厂架构师面试深挖题”的价值
- 是否能被代码证据支撑
- 是否能被候选人在三层追问后讲住
- 是否匹配 AI 应用架构 / Agent / RAG / Workflow / 存储 / 分布式 / 性能优化 JD

十、公司与 JD 口径映射

请用以下视角对亮点进行映射和重排。

1. 字节 / AI 应用 / RAG / Agent / 数据智能方向

重点看：

- 大模型应用架构
- RAG 端到端链路
- 多智能体 / workflow 编排
- Tool Use / Function Calling
- Prompt / Eval / DSL 体系
- AI 与业务系统、BI、低代码、数据智能的结合
- 线上效果稳定性、可观测性、评测与优化闭环
- 高并发场景下的推荐 / 搜索 / 内容链路
- 快速迭代能力和工程落地能力

2. 阿里 / 增长 / 平台 / 核心链路方向

重点看：

- 用户增长、营销活动、平台化能力
- 高并发、高可用、可运营、可灰度
- 系统长期迭代能力
- 交易 / 活动 / 互动链路的稳定性与治理
- 跨团队协作下的平台型抽象
- 配置化、策略化、可运营能力
- 业务系统智能化落地

3. 腾讯 / 稳定性 / 工程质量 / 平台支撑方向

重点看：

- 服务治理、质量体系、监报告警、统一异常
- 复杂业务长期维护与演进
- 中台化、组件化、可复用能力
- 权限、安全、审计、风控、平台规范
- 大规模系统下的稳态建设
- 工程质量、可维护性、风险控制

4. Google Staff / Senior+ 视角

重点看：

- 大规模分布式系统、检索、存储、网络、API、平台能力
- 模糊问题下的技术决策
- owning outcomes, 而不是只写模块代码
- 影响范围、抽象能力、长期演进能力
- 技术方向判断与 stakeholder 影响力
- 清晰的设计取舍和复杂度控制
- AI 应用架构的系统化抽象

5. Amazon Sr. SDE / SDE III 视角

重点看：

- customer obsession / ownership / invent and simplify
- 高可用、高扩展、高性能系统设计
- 端到端交付、设计评审、运维治理
- system design 可深挖性
- mentoring / best practices / operational excellence
- 可观测、可回滚、可运维
- AI 应用工程化和业务价值闭环

6. Binance Senior Backend / AI Backend 视角

重点看：

- 微服务、分布式系统、消息系统
- 监控、可观测性、实时性能诊断
- JVM / 线程池 / 网络 / I/O / DB / Redis / Kafka 等性能治理
- 高并发、高吞吐、快速演进场景下的工程落地
- AI 应用后端、聊天 / 实时链路、平台稳定性
- 交易、资产、风控、幂等、一致性能力

7. AI 应用架构师 JD 视角

重点看：

- 是否有 LLM 应用架构
- 是否有 RAG / Agent / Workflow
- 是否有 Prompt / Eval / Tool / DSL 体系
- 是否结合业务系统或数据分析场景
- 是否体现产品化落地
- 是否体现技术决策和项目推进
- 是否能带团队或指导成员

8. AI 性能优化架构师 JD 视角

重点看：

- 是否有工业级推理框架经验
- 是否有模型层优化
- 是否有通信层优化
- 是否有计算层优化
- 是否有全链路推理优化
- 是否有可验证量化成果
- 是否有 Benchmark / Profiler / 自动化调优

如果没有代码证据，请明确输出：

“不建议投递或主打 AI 性能优化架构师岗位；该项目更适合 AI 应用架构 / 后端平台 / RAG 工程化方向。”

9. 存储 / 分布式系统架构师 JD 视角

重点看：

- 是否有大规模分布式系统设计
- 是否有存储技术方向经验
- 是否有数据一致性、元数据、索引、缓存、容灾、迁移等能力
- 是否有系统规划和长期演进能力
- 是否有复杂问题攻坚和跨团队协作能力

如果代码证据不足，请明确输出：

“不建议硬写大规模分布式存储架构经验；可以保守表达为数据访问、检索、缓存或平台化工程能力。”

十一、输出结构

请按以下结构输出。

一、项目技术画像

用 300~500 字总结：

- 项目属于什么系统类型
- 主链路是什么
- 关键业务对象是什么
- 架构更偏一致性优先还是可用性 / 吞吐优先
- 当前成熟度处于什么层次
- 最适合突出哪些架构师岗位能力
- 与 AI 应用架构 JD 的匹配度
- 与 AI 性能优化 JD 的匹配度
- 与分布式 / 存储架构 JD 的匹配度
- 站在 5 年及以上架构师 JD 看，这个项目最大的价值和最大的短板分别是什么

- 如果作为主项目讲，推荐主打什么
- 如果作为次项目讲，推荐保守表达什么

要求：

- 必须基于代码事实
- 不确定的地方直接写“不确定”
- 不要强行包装成复杂分布式系统
- 不要把普通 LLM API 调用包装成模型推理优化

二、业务背景重建

分三部分输出。

1. 代码已明确体现的业务事实

每条格式：

- 业务事实：
- 证据：
 - 文件路径：
 - 类 / 方法：
 - 表 / 接口 / 配置：
- 可写入简历程度：
- 适配 JD：
- 风险边界：

2. 可强推导的业务诉求

每条格式：

- 推导诉求：
- 推导依据：
- 可写边界：
- 适配 JD：
- 不可夸大点：

3. 仍不确定的业务背景

每条格式：

- 不确定点：
- 当前缺少什么证据：
- 对应 JD 风险：
- 面试中应该如何保守表达：

三、系统全景扫描

请输出“架构全景图的文字版”。

必须包含：

- 模块划分
- 核心链路

- 核心表
- 缓存
- MQ
- 定时任务
- 外部依赖
- 稳定性与治理设施
- 安全与权限设施
- AI / RAG / Agent / Workflow / DSL 相关能力, 如有
- 模型调用 / 推理 / Eval / Benchmark 能力, 如有
- 存储 / 检索 / 元数据能力, 如有
- 工程质量设施

推荐格式:

用户请求 / 外部任务

- > Controller / Handler
- > Application Service
- > Domain Service / Strategy / Workflow / Agent
- > Repository / DAO / Client / Tool / Retriever
- > DB / Redis / MQ / VectorDB / LLM / Object Storage / 外部 API
- > 日志 / Trace / Metrics / Eval / 异常处理

如果项目没有某类设施, 请明确写“未发现足够证据”。

四、TOP 15 项目亮点, 按架构师简历价值排序

每个亮点必须按以下格式输出:

亮点 N: {标题}

1. 亮点级别:

- S / A / B
- S: 代码直接证明, 强烈建议写简历
- A: 代码较强支撑, 可写但措辞要克制
- B: 更适合面试延展, 不建议直接写成已落地成果

2. 架构师资深度层级:

- L1 / L2 / L3 / L4

3. 亮点分类:

- AI 应用架构
- RAG
- Agent
- Workflow
- Tool Use
- Prompt
- Eval
- DSL
- 无代码 / 低代码
- 数据分析 / BI
- 一致性
- 交易系统
- 资产安全
- 高并发
- 分布式
- 存储 / 检索

- 工程治理
- 性能优化
- 设计模式
- 数据平台
- 安全治理
- 可观测性
- 平台工程
- AI 性能优化

4. 结论归类:

- 【代码已实现】
- 【强推导亮点】
- 【可扩展设计】

5. 主导度判断:

必须从以下四类中选择:

- 负责设计并落地
- 参与核心设计与实现
- 负责模块实现与优化
- 参与治理与演进

如果代码不足以证明“主导”，禁止使用“主导架构设计”“负责整体架构”等表述。

6. 一句话简历表述:

要求:

- 专业
- 克制
- 可信
- 适合写进 5 年及以上架构师简历
- 不虚构量化数据
- 不硬蹭 AI 性能优化 / 分布式存储 JD

7. 技术拆解:

- 背景问题
- 方案设计
- 实现机制
- trade-off
- 收益与价值，若无证据不要写量化数据

8. 代码证据:

- 文件路径
- 类名 / 方法名 / 函数名
- 关键配置项
- 关键调用链
- 关键逻辑摘要

9. 为什么这是架构师亮点，而不是普通工程实现

10. 理论 / 实践映射:

- 可映射到哪些经典理论或成熟实践
- 为什么能映射
- 哪些理论不能硬套

11. JD 适配映射:

- AI 应用架构 JD 怎么看
- Agent / RAG / Workflow JD 怎么看
- AI 性能优化 JD 怎么看
- 分布式 / 存储架构 JD 怎么看

- 平台架构 JD 怎么看

12. 大厂口径映射:

- 字节会怎么看
- 阿里会怎么看
- 腾讯会怎么看
- Google 会怎么看
- Amazon 会怎么看
- Binance 会怎么看

13. 面试官追问点:

至少 3 个。

14. 候选人回答建议:

- 架构师怎么答
- 哪些回答会显得浅
- 如何体现技术决策与 trade-off
- 如何避免吹过头

15. 风险提示:

- 哪些地方不能夸大
- 哪些地方还缺证据
- 哪些表述写进简历会有风险

16. 反吹牛审查:

- 如果我是面试官, 我会用哪一个追问来验证这个点是真是假?
- 这个点一旦追问到第三层, 最容易暴露的薄弱环节是什么?

17. 能写 / 不要写:

- 能写的词:
- 不要写的词:

五、最适合写进架构师简历的 8 条

请从上面的内容中精选 8 条, 改写成可直接放在简历中的表述。

要求:

- 专业、克制、可信
- 不虚构数据
- 有 5 年及以上架构师风格
- 每条最好体现“业务问题 + 技术方案 + 架构价值”
- 严格区分主项目版本和次项目版本
- 严格区分 AI 应用架构、平台架构、分布式 / 存储架构、AI 性能优化是否适配

输出格式:

1. 5 年及以上架构师通用版

- bullet 1
- bullet 2
- bullet 3
- bullet 4

2. AI 应用架构师版

- bullet 1
- bullet 2
- bullet 3
- bullet 4

3. Agent / RAG / Workflow 架构师版

- bullet 1
- bullet 2
- bullet 3
- bullet 4

4. 平台架构师版

- bullet 1
- bullet 2
- bullet 3
- bullet 4

5. 分布式系统 / 存储架构师版

- bullet 1
- bullet 2
- bullet 3
- bullet 4

如果代码证据不足，请明确写：

“该项目不建议主打分布式存储架构师版本。”

6. AI 性能优化架构师版

如果代码证据不足，请明确写：

“该项目不建议主打 AI 性能优化架构师版本，未发现模型量化、算子优化、通信优化或推理框架优化证据。”

7. 主项目版本

- bullet 1
- bullet 2
- bullet 3
- bullet 4
- bullet 5
- bullet 6

8. 次项目版本

- bullet 1
- bullet 2
- bullet 3

六、最适合面试展开的 8 条

每条给出：

- 面试官可能怎么问
- 这道题他真正想考什么
- 候选人怎么答更像 5 年及以上架构师

- 哪些答案会显得浅
- 最佳深度边界在哪里
- 如果被追问代码位置，应该指向哪里
- 如果被追问指标，应该如何在没有数据时保守回答
- 如何映射到 AI 应用架构 JD
- 如何映射到平台 / 分布式 / 存储 JD
- 是否适合映射到 AI 性能优化 JD，如果不适合要明确说明

七、按目标公司和 JD 重排的亮点清单

请分别输出：

1. 更适合字节 AI / RAG / Agent / 数据智能岗的前 6 条

每条说明：

- 为什么适合
- 应该怎么写
- 不应该怎么写
- 是否体现 LLM / RAG / Agent / Eval / Tool / Workflow

2. 更适合阿里增长 / 平台 / 核心链路岗的前 6 条

每条说明：

- 为什么适合
- 应该怎么写
- 不应该怎么写
- 是否体现平台化、配置化、可运营、稳定性

3. 更适合腾讯稳定性 / 平台治理岗的前 6 条

每条说明：

- 为什么适合
- 应该怎么写
- 不应该怎么写
- 是否体现工程质量、治理、权限、安全、可维护性

4. 更适合 Google Staff / Senior+ 岗的前 6 条

每条说明：

- 为什么适合
- 应该怎么写
- 不应该怎么写
- 是否体现抽象、复杂系统、长期演进、技术决策

5. 更适合 Amazon Sr. SDE / SDE III 岗的前 6 条

每条说明：

- 为什么适合
- 应该怎么写
- 不应该怎么写
- 是否体现 ownership、operational excellence、端到端交付

6. 更适合 Binance Senior Backend / AI Backend 岗的前 6 条

每条说明：

- 为什么适合

- 应该怎么写
- 不应该怎么写
- 是否体现高并发、微服务、可观测性、实时诊断、AI 应用后端

7. 更适合 AI 应用架构师 JD 的前 6 条

每条说明：

- 对应 JD 哪一条
- 代码证据是什么
- 简历怎么写
- 面试怎么讲
- 风险边界是什么

8. 更适合 AI 性能优化架构师 JD 的前 6 条

如果代码证据不足，请不要强行列 6 条。

可以输出：

“未发现足够代码证据支持 AI 性能优化架构师方向。以下仅为可扩展设计或岗位差距分析。”

然后列：

- 当前缺什么
- 如果要补，需要补在哪些模块
- 补完后能形成什么简历亮点
- 现在不能写成已实现的原因

9. 更适合分布式 / 存储架构师 JD 的前 6 条

如果代码证据不足，请明确说明：

“该项目更适合作为数据访问 / 检索 / 平台工程项目，不建议包装成大规模分布式存储系统。”

八、定向改写：最匹配的 6 条核心亮点

基于用户指定或默认的：

- 目标公司
- 目标岗位
- 候选人定位：5 年及以上架构师
- 项目定位
- 语言风格
- 参考 JD

重新筛选最匹配的 6 条亮点。

每条按以下格式输出：

亮点 1：{亮点标题}

1. 亮点分级

- 亮点等级：S / A / B
- 架构师资深度层级：L1 / L2 / L3 / L4
- 适配岗位：
- 适配 JD：
- 结论归类：【代码已实现】 / 【强推导亮点】 / 【可扩展设计】
- 主导度判断：

- 负责设计并落地 / 参与核心设计与实现 / 负责模块实现与优化 / 参与治理与演进
- 判断依据：只能引用已确认代码事实

2. 简历表述

5 年及以上架构师版本

要求：

- 克制
- 架构化
- 偏系统设计、链路治理、技术决策
- 不夸大主导度

AI 应用架构师版本

要求：

- 仅当项目涉及 LLM / RAG / Agent / Tool / Workflow / Prompt / Eval / DSL 时输出
- 不把普通接口调用写成复杂 AI 架构
- 不把简单 Prompt 写成完整 Prompt 工程体系

平台架构师版本

要求：

- 强调平台化、配置化、可扩展、可治理、可复用
- 不把普通后台管理系统硬写成平台架构

分布式 / 存储架构师版本

要求：

- 仅当代码涉及存储、检索、元数据、分布式一致性、大规模数据链路时输出
- 证据不足时必须保守

AI 性能优化架构师版本

要求：

- 仅当代码涉及模型推理优化、算子、通信、GPU、量化、benchmark、profiler 时输出
- 否则明确“不适合写成该方向已实现经历”

3. 为什么适合目标公司和 JD

只输出与本次目标公司和 JD 强相关的内容，不要泛泛罗列。

4. 高频追问

1. 追问 1
2. 追问 2

5. 反吹牛审查

- 如果我是面试官，我会用哪一个追问验证这个点是真是假？
- 这个点一旦追问到第三层，最容易暴露的薄弱环节是什么？

6. 能写 / 不要写

能写的词

例如：

- 参与设计

- 实现
- 优化
- 治理
- 抽象
- 接入
- 改造
- 沉淀
- 统一封装
- 局部优化
- 核心模块实现
- 核心链路治理
- AI 应用工程化
- RAG 链路建设
- Workflow 编排
- Tool 调用封装
- 数据访问优化
- 检索链路优化

不要写的词

例如：

- 主导整体架构
- 千万级 QPS
- 零故障
- 平台化闭环
- 全链路治理
- 业界领先
- 负责整体架构
- 从 0 到 1 搭建全平台
- 主导工业级分布式推理框架
- 主导模型量化优化
- 主导 RDMA 通信优化
- 主导 CUDA 算子优化
- 主导大规模分布式存储系统

除非代码和数据明确证明。

7. 风险提示

- 可写边界：
- 不可夸大点：
- JD 匹配风险：
- 面试准备重点：

九、S 级核心亮点专项文档

本章节只整理 S 级亮点。

特别要求：

S 级文档必须保证“刚刚接手项目的小白也能看懂”。

因此每个 S 级亮点必须使用以下结构，不允许只写抽象架构词。

S 级亮点总览

	排名	亮点标题	代码事实强度	简历价值	面试深挖价值	适配岗位	适配 JD	推荐写法
	---	---	---	---	---	---	---	---
	1	{标题}	强 / 中 / 弱	高 / 中 / 低	高 / 中 / 低	{岗位}	{JD}	{一句话}

S1: {亮点标题}

1. 这个亮点一句话说人话

要求:

- 用非技术同学也能理解的话解释
- 不超过 100 字
- 不堆术语

示例格式:

“这个模块主要解决的是: 当用户发起 xxx 操作时, 系统需要在 xxx、xxx、xxx 之间保证流程正确, 避免 xxx 问题。”

2. 业务背景, 小白版

要求回答:

- 这个功能是给谁用的?
- 用户 / 系统在这里想完成什么事?
- 如果这个功能出问题, 会造成什么后果?
- 为什么它不是普通 CRUD?
- 在整个项目里, 它处于什么位置?
- 它为什么适合 5 年及以上架构师简历?

如果业务背景不确定, 必须写:

“代码中没有足够证据证明真实业务场景, 只能根据接口 / 命名 / 表结构推导为.....”

3. 核心术语解释

把本亮点涉及的术语解释清楚。

格式:

	术语	小白解释	在代码中的体现
	---	---	---
	幂等	多次请求只生效一次, 避免重复创建 / 重复扣减	xxx
	缓存	把常用数据放到 Redis / 本地内存, 减少反复查 DB	xxx
	RAG	先查知识库, 再把查到的内容交给大模型回答	xxx
	Agent	能根据目标自主决定下一步调用哪个工具或流程的程序	xxx
	Workflow	把多个步骤按顺序或条件编排起来执行	xxx
	Eval	用一套样本和指标评估模型或系统效果	xxx
	DSL	给业务配置或流程表达用的小型规则语言	xxx

要求:

- 每个术语都必须配“小白解释”
- 不允许只写英文缩写
- 不允许默认读者懂分布式 / RAG / MQ / 事务 / AI 推理

4. 代码入口在哪里

必须写清楚：

- 用户请求从哪个 Controller / Handler 进入？
- 对应接口路径是什么？
- 调用哪个 Service？
- Service 再调用哪些 Domain / Repository / Client？
- 是否调用 LLM / VectorDB / MQ / Redis / DB / 外部 API？
- 最后结果返回到哪里？

推荐格式：

入口接口：

```
{Controller}.{method}
-> {Service}.{method}
   -> {Domain / Strategy / Workflow / Agent}.{method}
       -> {Repository / DAO / Tool / Retriever / LLM Client}.{method}
           -> {DB / Redis / MQ / LLM / VectorDB / Object Storage}
```

如果无法确认完整链路，必须写：

“当前代码只能确认到 xxx，后续链路未发现足够证据。”

5. 核心代码文件清单

文件路径	类 / 方法	作用	小白理解	建议阅读顺序
xxx	xxx	xxx	这里相当于入口	1
xxx	xxx	xxx	这里处理核心业务规则	2
xxx	xxx	xxx	这里负责存取数据	3

要求：

- 不只列路径，还要解释每个文件“干什么”
- 让新人知道从哪里开始读代码
- 标出建议阅读顺序

6. 主链路逐步解释

用步骤解释，不要只给架构图。

格式：

1. 第一步：用户 / 系统触发 xxx
 - 代码位置：
 - 做了什么：
 - 为什么要做：
2. 第二步：系统校验 xxx
 - 代码位置：
 - 做了什么：
 - 为什么要做：
3. 第三步：系统执行核心逻辑 xxx
 - 代码位置：
 - 做了什么：
 - 为什么要做：
4. 第四步：系统写入 / 查询 xxx
 - 代码位置：

- 做了什么：
- 为什么要做：

5. 第五步：系统返回 / 异步处理 xxx

- 代码位置：
- 做了什么：
- 为什么要做：

7. 数据是怎么流动的

必须解释：

- 输入数据从哪里来
- 中间经过哪些对象转换
- 写入哪些表 / 缓存 / MQ / 向量库 / 对象存储
- 返回给调用方什么
- 哪些字段是关键字段
- 如果涉及 AI, Prompt / Context / Tool Result / Model Response 如何流动
- 如果涉及 RAG, Query / Chunk / Embedding / Recall / Rerank / Answer 如何流动

推荐格式：

Request DTO

- > Command / BO
- > Domain Model
- > DO / Entity
- > DB / Redis / MQ / VectorDB / LLM
- > Response VO

如果项目没有清晰分层，也要如实说明：

“当前代码中 DTO / Domain / DO 分层不明显，更多是 xxx 直接传递。”

8. 为什么这样设计

要求用小白能懂的话解释：

- 为什么要分这些层？
- 为什么要用缓存 / MQ / 事务 / 策略 / 状态机 / Workflow / Agent / RAG？
- 如果不用这个方案，会有什么问题？
- 当前方案解决了什么问题？
- 当前方案牺牲了什么？
- 这个设计如何体现架构师能力？

9. Trade-off

至少写 3 条：

- 性能 vs 一致性
- 简单实现 vs 可扩展性
- 同步处理 vs 异步处理
- 实时性 vs 吞吐
- 缓存性能 vs 数据一致性
- 模型效果 vs 调用成本
- RAG 召回完整性 vs 响应速度
- Agent 自主性 vs 可控性
- Workflow 灵活性 vs 调试复杂度
- 代码复杂度 vs 长期维护
- 存储成本 vs 查询性能

要求：

不能只写“提升性能”，必须写“代价是什么”。

10. 异常场景和兜底

说明：

- 哪些地方可能失败？
- DB 失败怎么办？
- Redis 失败怎么办？
- MQ 失败怎么办？
- 外部 API / LLM 失败怎么办？
- Tool 调用失败怎么办？
- Workflow 某一步失败怎么办？
- RAG 检索为空怎么办？
- 模型返回不稳定怎么办？
- 重复请求怎么办？
- 并发请求怎么办？
- 部分成功怎么办？
- 是否有补偿？
- 是否有重试？
- 是否有人工处理入口？

如果代码没有实现，必须明确写：

“未发现足够证据，不能写成已实现；可以作为后续优化方向。”

11. 这个亮点为什么值得写架构师简历

必须说明：

- 它解决了什么真实工程问题？
- 它比普通 CRUD 高级在哪里？
- 它能体现候选人什么架构师能力？
- 它适合投哪些岗位？
- 它不适合投哪些岗位？
- 它与 AI 应用架构 JD 的关系是什么？
- 它与分布式 / 存储 / AI 性能优化 JD 的关系是什么？

12. 简历表述

分别输出：

克制专业版

更偏架构版

更偏业务结果版

AI 应用架构师版

平台架构师版

分布式 / 存储架构师版，如适用

AI 性能优化架构师版，如适用

要求：

- 不虚构数据
- 不夸大主导度
- 不写没有证据的规模
- 不把应用层优化写成模型底层优化

13. 面试三层追问准备

第一层：你做了什么？

给出候选人回答建议。

第二层：为什么这么设计？

给出候选人回答建议。

第三层：如果流量、数据量、失败场景扩大，怎么处理？

给出候选人回答建议。

14. 面试官验证问题

- 如果我是面试官，我会怎么判断你是不是真的做过？
- 我会问哪段代码？
- 我会问哪个异常场景？
- 我会问哪个设计取舍？
- 我会问有没有指标时，你应该怎么保守回答？
- 我会如何判断你只是会调 API，还是理解 AI 应用架构？
- 我会如何判断你是否真的做过 RAG / Agent / Workflow？
- 我会如何判断你是否真的做过 AI 性能优化？
- 我会如何判断你是否真的做过分布式 / 存储架构？

15. 第三层追问最容易暴露的薄弱环节

例如：

- 说不清事务边界
- 说不清缓存一致性
- 说不清 MQ 幂等
- 说不清线程池隔离
- 说不清状态机状态流转
- 说不清 RAG 检索质量如何评估
- 说不清 LLM 调用失败怎么降级
- 说不清 Agent Tool 权限边界
- 说不清 Workflow 失败恢复
- 说不清 Prompt 版本管理
- 说不清 Eval 指标
- 说不清慢查询怎么定位
- 说不清存储读写路径
- 说不清模型推理优化和 LLM API 调用的区别

16. 新人接手阅读路线

请给一个“小白接手项目后如何读懂这个亮点”的阅读路线：

1. 先看哪个 README / 配置
2. 再看哪个 Controller / Handler
3. 再看哪个 Service
4. 再看哪个 Domain / Strategy / Workflow / Agent

5. 再看哪个 Repository / DAO / Tool / Retriever / LLM Client
6. 再看哪些 DB / Redis / MQ / VectorDB / Object Storage 配置
7. 最后看哪些测试 / 脚本 / 配置

每一步说明：

- 看什么
- 为什么看
- 看完应该理解什么

17. 风险边界

- 不能说：
- 可以说：
- 需要补充证据后才能说：
- 不适合写进简历的夸大版本：
- 不适配哪个 JD：
- 面试中需要主动降调的地方：

十、系统优化专项文档

请额外输出一份【系统优化专项文档】。

该文档当作一个独立的优化文档，用来指导后续代码优化、简历包装和面试展开。

系统优化点必须严格分为三类：

1. 【当前代码已实现，可写入简历】
2. 【当前代码有迹象，只能克制表达】
3. 【当前未实现，仅作为后续优化方案】

重点关注：

- 线程串行改并行
- 网络 IO 优化
- 请求折叠 / SingleFlight
- 核心链路缓存引入
- 批量查询
- 慢 SQL 优化
- 异步化
- 热点保护
- RAG / LLM 调用优化
- Prompt 缓存
- 上下文裁剪
- Tool 调用治理
- Workflow 执行优化
- Eval 自动化
- 可观测性治理
- 线程池治理
- 连接池治理
- 超时 / 重试 / 熔断 / 降级
- 大对象裁剪
- 序列化优化
- 数据库索引优化
- MQ 消费幂等
- 任务调度优化
- 存储读写路径优化

- 检索链路优化

如果代码涉及 AI 推理框架，还要额外关注：

- batching
- continuous batching
- KV Cache
- prefix cache
- speculative decoding
- quantization
- sparse attention
- MoE routing
- communication overlap
- benchmark
- profiler

如果代码没有相关证据，必须写成【可扩展设计】，不能写成已实现。

0. 系统优化文档边界

本章节只讨论工程优化点，包括但不限于：

- 串行链路改并行
- 网络 IO 优化
- 请求折叠 / 合并请求
- 核心链路缓存
- 批量 SQL / 批处理
- 连接池 / 线程池治理
- 异步化 / MQ 解耦
- 超时 / 重试 / 熔断 / 降级
- 慢查询治理
- 数据结构与序列化优化
- 大对象 / 大响应裁剪
- 热点 Key / 热点数据保护
- RAG 检索链路优化
- LLM 调用成本与延迟优化
- Agent / Workflow 执行优化
- Tool 调用安全与性能治理
- 存储 / 检索读写路径优化
- 模型推理链路优化，如有证据

必须严格区分：

- 【当前已实现】：代码已有明确证据，可写简历
- 【强推导可写】：代码有部分证据，措辞必须克制
- 【建议补齐】：当前未实现，只能作为优化方案

1. 系统优化点总览表

序号	优化方向	当前状态	证据强度	简历可写性	面试价值	JD 适配	推荐级别
1	串行改并行	已实现 / 未实现 / 待证实	强 / 中 / 弱	可写 / 谨慎写 / 不建议写	高 / 中 / 低	xxx	S / A / B

| 2 | 请求折叠 | 已实现 / 未实现 / 待证实 | 强 / 中 / 弱 | 可写 / 谨慎写 / 不建议写 | 高 / 中 / 低 | xxx | S / A / B |
| 3 | 核心链路缓存 | 已实现 / 未实现 / 待证实 | 强 / 中 / 弱 | 可写 / 谨慎写 / 不建议写 | 高 / 中 / 低 | xxx | S / A / B |

2. 单个优化点文档模板

优化点 N: {优化标题}

1. 优化类型

- 性能优化 / 稳定性优化 / 成本优化 / 可用性优化 / 工程治理优化 / AI 应用链路优化 / 存储检索优化 / 推理优化
- 适配岗位: AI 应用架构师 / 平台架构师 / 分布式系统架构师 / 存储架构师 / AI 性能优化架构师
- 当前状态: 【当前已实现】 / 【强推导可写】 / 【建议补齐】
- 简历推荐级别: S / A / B

2. 原始问题

说明优化前可能存在的问题:

- 是否存在串行等待?
- 是否存在重复请求?
- 是否存在高频 DB 查询?
- 是否存在外部服务 IO 阻塞?
- 是否存在慢查询 / 大分页 / N+1 查询?
- 是否存在 LLM / embedding / rerank 调用成本高或耗时长?
- 是否存在 Tool 调用不受控?
- 是否存在 Workflow 某一步失败导致整体失败?
- 是否存在 RAG 召回慢或结果不稳定?
- 是否存在存储读写路径过长?
- 是否存在推理服务吞吐瓶颈?

注意:

如果代码没有明确证据, 只能写“从代码结构看存在优化空间”, 不能写“线上曾出现问题”。

3. 当前代码事实

- 文件路径:
- 类 / 方法:
- 调用链:
- 数据库 / Redis / MQ / 外部服务:
- LLM / VectorDB / Tool / Workflow:
- 关键代码摘要:

4. 优化方案

说明具体怎么做, 而不是只写概念。

示例: 串行改并行

- 将多个互不依赖的外部服务调用从串行执行改为 `CompletableFuture` / `goroutine` / `async task` 并行编排
- 设置统一超时、失败兜底和结果聚合
- 对强依赖结果保留 `fail-fast`, 对弱依赖结果允许降级

示例: 请求折叠

- 对同一资源 ID / query / cache key 的短时间重复请求进行 singleflight 合并
- 首个请求负责加载数据，后续请求等待同一个 Future / promise / channel
- 避免热点场景下重复打 DB / 外部 API / LLM

示例：核心链路缓存

- 对读多写少、变更频率低的数据引入 Redis / 本地缓存
- 设置 TTL、主动失效、版本号或双删策略
- 对热点数据增加本地缓存或预热机制

示例：RAG 链路优化

- 对 query rewrite、召回结果、rerank 结果进行缓存
- 控制 chunk 数量和上下文长度
- 对空召回、低置信召回做兜底
- 增加检索日志和 Eval 数据闭环

示例：Workflow 执行优化

- 将无依赖节点并行化
- 为每个节点设置超时、重试和降级
- 保存节点执行状态，支持失败恢复
- 记录节点输入输出，便于排查问题

示例：AI 推理优化

仅在代码有证据时使用：

- 引入 batching / continuous batching
- 使用 KV Cache / prefix cache
- 做 speculative decoding
- 做量化或稀疏化
- 建立 benchmark 与 profiler 体系

5. Trade-off

- 并行化带来的线程池压力 / 上下文传播问题
- 缓存带来的一致性问题
- 请求折叠带来的等待放大 / 单点慢请求问题
- 异步化带来的最终一致性和补偿复杂度
- 批处理带来的实时性下降
- 重试带来的请求放大风险
- 降级带来的结果完整性下降
- Agent 自主性增强带来的可控性下降
- RAG 召回更全带来的延迟上升
- 模型成本下降可能带来的效果下降
- 推理吞吐优化可能带来的精度或资源权衡

6. 可写简历表述

如果当前已实现

可以写成：

“围绕核心链路性能瓶颈，完成 xxx 优化，通过 xxx 机制降低重复 IO / 减少串行等待 / 提升链路稳定性。”

如果只是建议补齐

不能写进已完成经历，只能写成：

“针对 xxx 链路设计了缓存 / 请求折叠 / 并行编排优化方案，作为后续性能治理方向。”

7. 高频追问

1. 为什么这个地方能并行？依赖关系怎么判断？
2. 缓存一致性怎么保证？
3. 请求折叠如果首个请求很慢怎么办？
4. 线程池参数怎么设置？
5. 超时、重试、降级怎么配合？
6. 有没有压测或指标证明？
7. RAG 效果怎么评估？
8. Agent / Tool 调用失败怎么兜底？
9. Workflow 节点失败怎么恢复？
10. 如果说 AI 性能优化，具体优化的是模型层、通信层还是计算层？

8. 反吹牛审查

- 面试官验证问题：
 - 例如：你把哪几个调用从串行改成并行？调用链和代码位置在哪？
- 第三层最容易暴露的问题：
 - 例如：说不清线程池隔离、超时传播、异常聚合、缓存失效策略、请求折叠的 key 设计、RAG Eval 指标、推理优化边界。

9. 能写 / 不要写

能写

- 优化核心链路
- 降低重复请求
- 减少不必要 IO
- 引入缓存降低数据库访问压力
- 对独立调用进行并行编排
- 增加超时和降级保护
- 完善 RAG 链路观测
- 优化 LLM 调用成本与稳定性
- 增强 Workflow 执行可靠性

不要写

- QPS 提升 xx%
- 延迟降低 xx%
- 支撑千万级并发
- 彻底解决性能问题
- 零故障
- 全链路性能治理
- 主导模型推理优化
- 主导 INT4 / INT8 / FP8 量化
- 主导 RDMA 通信优化
- 主导 CUDA / Triton 算子优化

除非代码证据和压测数据明确支持。

十一、当前代码还缺但最值得补充的 10 条

要求：

- 明确补在哪个模块
- 为什么值得补
- 补完后能形成什么新的架构师简历亮点
- 更适合哪类大厂岗位
- 更适合哪个 JD
- 如果可能，给出建议修改点、伪代码位置或设计草图
- 必须明确标注为【可扩展设计】
- 禁止写成当前已实现

每条格式：

建议补齐 N: {标题}

- 当前状态：【可扩展设计】
- 建议补充模块：
- 当前代码已有基础：
- 当前缺口：
- 为什么值得补：
- 补完后的简历亮点：
- 适配目标公司：
- 适配 JD：
- 设计草图：
- 伪代码位置：
- 风险和代价：
- 不能现在写成已实现的原因：

必须至少覆盖以下方向中的若干项：

1. RAG Eval 评测体系
2. Prompt 模板版本管理
3. Tool 调用权限与审计
4. Workflow 节点级状态与恢复
5. LLM 调用超时 / 重试 / 降级
6. 检索链路缓存与请求折叠
7. 文档解析 / embedding / 建索引异步化
8. 核心链路可观测性
9. 多租户 / 数据权限隔离
10. 性能 benchmark / 压测体系
11. 存储 / 检索链路优化
12. AI 推理优化能力建设，如项目适合

十二、JD 差距分析

请新增一节，专门分析当前项目与参考 JD 的匹配度。

1. AI 应用架构师 JD 匹配度

输出：

- 已匹配能力：
- 部分匹配能力：
- 未发现证据的能力：
- 简历主打建议：
- 面试风险：
- 补齐建议：

2. Agent / RAG / Workflow JD 匹配度

输出:

- 已匹配能力:
- 部分匹配能力:
- 未发现证据的能力:
- 简历主打建议:
- 面试风险:
- 补齐建议:

3. AI 性能优化架构师 JD 匹配度

输出:

- 已匹配能力:
- 部分匹配能力:
- 未发现证据的能力:
- 是否建议主打该 JD:
- 如果不建议, 原因是什么:
- 如果想补齐, 需要补哪些工程能力:

特别注意:

如果没有模型量化、通信优化、算子优化、GPU / AI 芯片、Benchmark / Profiler 证据, 必须明确写:
“当前项目不建议主打 AI 性能优化架构师岗位。”

4. 分布式 / 存储架构师 JD 匹配度

输出:

- 已匹配能力:
- 部分匹配能力:
- 未发现证据的能力:
- 是否建议主打该 JD:
- 如果不建议, 原因是什么:
- 如果可迁移, 应该怎么表达:

十三、简历风险审查

请专门做一节, 指出:

- 哪些技术点容易被候选人误写大
- 哪些说法面试时一追问就会露馅
- 哪些只能说“参与设计 / 实现”, 不能说“主导架构”
- 哪些更适合写成“优化 / 治理”, 不适合写成“从 0 到 1 架构设计”
- 哪些属于普通工程实现, 不适合当核心亮点
- 哪些词汇高风险
- 哪些亮点适合简历, 哪些只适合面试延展
- 哪些点需要补充代码或数据后才能写
- 哪些 JD 方向不能硬投或硬写

请输出:

1. 高风险表述

高风险表述	风险原因	推荐替代表述

2. 容易被追问露馅的点

技术点	面试官会怎么问	容易暴露的问题	防守方式

3. 主导度边界

亮点	推荐主导度表述	禁止表述	原因

4. JD 风险边界

JD 方向	是否适合主打	风险原因	推荐表达

5. 简历可写边界

可写	谨慎写	不建议写

十四、1 分钟项目介绍版本

要求：

- 不超过 250 字
- 面向 5 年及以上架构师岗位
- 只讲最核心业务、主链路、本人贡献、技术价值
- 不堆名词
- 不虚构规模和数据
- 适合面试开场白

输出五个版本：

1. 克制专业版

2. 更偏架构版

3. 更偏业务结果版

4. AI 应用架构师版

5. 平台 / 分布式架构师版

如果 AI 性能优化证据不足，不要输出 AI 性能优化版，改为说明：
“当前项目不适合用 1 分钟介绍主打 AI 性能优化。”

十五、5 分钟深挖版本

按以下结构讲：

1. 项目背景

2. 系统定位
3. 核心链路
4. 我参与 / 负责的模块
5. 最核心的 2-3 个技术难点
6. 关键设计取舍
7. AI 应用 / RAG / Agent / Workflow 能力, 如有
8. 系统优化与治理
9. 风险边界和后续演进
10. 与目标 JD 的匹配点

要求:

- 不能写成背诵稿, 要像面试中自然讲项目
- 每个技术点后面都要能接住追问
- 对不确定的地方要保守表达
- 对没有指标的数据不要虚构
- 对 AI 性能优化、存储架构等 JD 方向要严格基于代码证据

十六、最终结论

请给出:

1. 这个项目最值得打的 3 张牌
2. 最不该硬吹的 3 个点
3. 如果定位为 5 年及以上架构师, 应该怎么包装
4. 如果目标是 AI 应用架构师, 应该怎么包装
5. 如果目标是 Agent / RAG / Workflow 架构师, 应该怎么包装
6. 如果目标是平台架构师, 应该怎么包装
7. 如果目标是分布式 / 存储架构师, 是否适合, 如何保守表达
8. 如果目标是 AI 性能优化架构师, 是否适合, 为什么
9. 如果只能选 3 条写进简历, 推荐哪 3 条
10. 如果只能选 1 条作为项目主讲点, 推荐哪 1 条
11. 如果需要补代码, 最优先补哪 3 个优化点
12. 该项目最推荐投递的岗位方向排序

十七、最终自检

在输出前, 请逐条检查:

1. 这条亮点是否能被代码事实直接支撑?
2. 如果面试官问“代码在哪里”, 是否能回答?
3. 如果面试官追问“为什么这么设计”, 是否能讲清 trade-off?
4. 如果面试官追问“有没有指标”, 是否存在真实证据?
5. 如果没有指标, 是否已经避免写百分比和规模数据?
6. 是否错误使用了“主导”“整体架构”“全链路”“高并发”“千万级”等高风险词?
7. 是否错误使用了“主导模型推理优化”“量化优化”“RDMA”“CUDA 算子优化”等无证据词?
8. 是否把建议补齐的优化点写成了已实现?
9. 是否符合 5 年及以上架构师定位?
10. 是否给出了每个亮点第三层追问的薄弱环节?
11. 是否把 s 级亮点整理成了可复述、可面试、可写简历的专项文档?
12. s 级亮点文档是否能让一个刚刚接手项目的小白看懂?
13. 是否解释了核心术语?
14. 是否给出了代码阅读顺序?
15. 是否说明了入口、主链路、数据流和异常场景?

16. 是否明确标注了哪些是已实现, 哪些只是建议补齐?
17. 是否对 AI 应用架构 JD 做了匹配分析?
18. 是否对 AI 性能优化 JD 做了风险隔离?
19. 是否对分布式 / 存储架构 JD 做了保守判断?
20. 是否避免把普通 LLM API 调用包装成模型底层优化?

十八、特别注意

- 先扫描、再判断、再排序, 不要一上来只盯着 controller / service。
- 对每个亮点都要问自己: 这是普通实现, 还是能打 5 年及以上架构师面试的点?
- 对每个简历表述都要问自己: 这句话面试追问 3 层后, 候选人还能讲住吗?
- 如果没有证据支撑, 请直接说“没有发现足够证据”。
- 如果项目更偏普通业务系统, 也请如实输出, 不要强行包装成复杂分布式系统。
- 如果项目只是 LLM API 调用, 不要包装成模型推理优化。
- 如果没有量化、算子、通信、GPU、Benchmark、Profiler, 不要写 AI 性能优化架构经验。
- 如果没有分布式存储、元数据、副本、一致性、数据分片等证据, 不要写大规模分布式存储架构经验。
- 不要为了显得高级而硬套 TCC、Saga、Raft、多活、单元化、CQRS、Event Sourcing、RDMA、CUDA、Triton、MoE、Speculative Decoding 等概念。
- 这是为了做“可信、能讲、能落地”的简历亮点与面试准备, 不是为了写架构论文。
- S 级亮点文档必须像“新人接手说明书”, 不能只像“架构师汇报材料”。
- 系统优化专项文档必须明确哪些能写进简历, 哪些只是后续优化建议。
- 所有输出都要克制、专业、可验证。